1 Stochastic Model and Quality Control

This document covers the *measurement* side of the estimator's error budget: the strategies that populate the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that validates an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ produced here is the block-diagonal skeleton of the observation-equations noise model; this document supplies the per-strategy variance interiors of that skeleton. The QC layer is built on the same post-fit residual statistics the filter already computes for outlier rejection, but reframes them as the classical geodetic detection theory of Baarda and Teunissen [3].

Two scopes are deliberately *out* of this document and only cross-referenced. First, the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA $\boldsymbol{R}$-inflation that react to a detected blunder live in the Kalman-filter gating section — this document explains how the QC statistics feed an *alternative* gate but does not re-derive the dispatch. Second, the per-state process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias) is purely a *dynamics*-side quantity and lives with the smoother (the process-noise section); everything below is measurement noise only.

1.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch lives in `src/core/model/StochasticModel.cpp`; each variant returns a variance in m^2 that becomes one diagonal entry of $\boldsymbol{R}$. The three geometry-aware strategies share a common low-elevation guard: when $\sin\theta \le 10^{-6}$ the elevation amplification is suppressed and the un-amplified variance is returned, so a satellite at the horizon does not inflate to a numerically infinite weight (`computeElevation`). The `Constant` strategy has no elevation term at all — it returns its a-priori variance directly from the dispatch (`computeConstant`) and never reaches the guard.

(a) Constant. The trivial strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{1}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). This is the reference against which the other strategies are multiplicative.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = 1.0,\ \ p_\Phi = 0.5.} \tag{2}$$

The two distinct powers are the key design choice (`computeElevation`, `src/core/model/StochasticModel.cpp`): with $p_P = 1$ the code variance follows the legacy $\csc^2\theta$ law, whereas phase uses the gentler $p_\Phi = 0.5$ giving a $\csc\theta$ *variance* ($\csc^{1/2}\theta$ in σ). Carrier-phase tracking degrades far more slowly with elevation than code multipath does, and the half-power keeps low-elevation phase from being over-deweighted into irrelevance. The exponent in code is $-2p_{\mathrm{type}}$ because the strategy

works in variance, not σ: $\left(\sin\theta\right)^{-2p}$ is exactly $\left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` (i.e. everything but carrier phase, including Doppler) take the code power.

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [1]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{3}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). The *variance*-additive composition in Equation (3) is mathematically the correct combination of two independent noise sources. The legacy σ-additive alternative $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$ is explicitly rejected in `computeSnr` (lines 170–181) because squaring it produces an unphysical $2ab\,10^{-\mathrm{C/N_0}/10}$ cross-term that has no noise-process interpretation — it would couple the two contributions as if they were correlated when they are not.

When no `S...` observable is present the model does not switch functional form; instead it *synthesises* a density from elevation so the noise curve stays continuous across the boundary (`synthesizeCn0FromElevation`, lines 220–225):

$$\mathrm{C/N_0}(\theta) = 50 - 15\left(1 - \max(0, \sin\theta)\right) \ \ [\mathrm{dB\,Hz}], \tag{4}$$

i.e. a 50 dB Hz zenith maximum dropping 15 dB to the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow: on code rows a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR} (wider-bandwidth signals such as L5/E5a get $\sim\sqrt{10}$ less correlator-noise σ, hence a factor $\sim 1/10$ in variance), while phase stays band-flat because PLL noise is governed by carrier-loop bandwidth, not chip rate. Finally the Wieser hybrid step pipes the SNR variance through Equation (2) so that elevation-correlated multipath — which pure SNR weighting under-weights, since the carrier loop tracks cleanly even while the range is corrupted — is still captured.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`, lines 231–345):

$$\boxed{\sigma^2_{r,i,s} = \left(\sigma_{\mathrm{block}(i),\,\mathrm{type}}\; s_{\mathrm{band}}\; s_{\mathrm{attr}}\right)^2 \left(\sin\theta\right)^{-2p_{\mathrm{type}}}.} \tag{5}$$

Here $\sigma_{\mathrm{block}(i),\mathrm{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m` when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because RINEX-3 attributes carry constellation-specific meaning (a `C` attribute is the noisy GPS C/A but the high-precision Galileo E1C pilot). The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase carry separate band/attribute tables (`SatBlockNoiseParams::ObservableScale`) since chip-rate calibration governs code while carrier-tracking calibration — a much smaller band gradient — governs phase. As with the other geometry-aware strategies the block variance is finally amplified by Equation (2); phase only reaches this path when `apply_to_phase` is set, otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies populate diagonal entries only; the undifferenced $\boldsymbol{R}$ is therefore block-diagonal, exactly the skeleton of the observation noise model. Structured off-diagonal correlation appears only after differencing (Section 1.5).

1.2 A-posteriori variance factor

Once an epoch has been updated, the post-fit residuals $\hat{\boldsymbol{v}}$ provide an internal-consistency check on whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\mathrm{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^2}{R_{ii}} \Bigg|_{\boldsymbol{R}\,\mathrm{diag}}, \qquad \mathrm{dof} = \mathrm{tr}\,\boldsymbol{\Theta},} \tag{6}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (11) below), i.e. the *effective* redundancy of a recursively-filtered epoch rather than the naive $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanFilter.cpp`). Because $\boldsymbol{R}$ is carried in absolute units, the a-priori variance of unit weight is exactly $\sigma_0^2 = 1$; a well-scaled model therefore yields $\hat{\sigma}_0^2 \approx 1$, with $\hat{\sigma}_0^2 \gg 1$ signalling under-weighted (optimistic) noise and $\hat{\sigma}_0^2 \ll 1$ pessimistic noise.

The variance factor is a *reporting-only* rescale: it never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter emits a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$, i.e. $\sigma_{\mathrm{pos}} \to \sigma_{\mathrm{pos}} \sqrt{\hat{\sigma}_0^2}$, leaving the filter state untouched (`validatePostFitResiduals`, `src/core/filter/KalmanFilter.cpp`).

The same quantity has a batch incarnation in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::run`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`src/core/filter/NeqAccumulator.cpp` lines 255–271)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \mathrm{dof} = n_{\mathrm{obs}} - (n_{\mathrm{params}} + n_{\mathrm{local}}), \tag{7}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\mathrm{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{8}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution, the Schur-reduced local parameters n_{local} are subtracted from the redundancy alongside the retained parameters n_{params}, and each retained parameter's formal σ is the $\hat{\sigma}_0^2$-scaled square root of the inverse normal matrix diagonal — the batch analogue of Equation (6)'s recursive dof.

Neither of these is the network producer's *operational* variance factor. The undifferenced network estimator does not form Ω/dof at all; it estimates a robust unit-weight σ as a scaled median-absolute-deviation over the weighted phase-leveling residual rows $\tilde{v}_i = |\hat{v}_i| \sqrt{w_i}$,

$$\hat{\sigma}_{0,\mathrm{net}} = \max\!\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}_i\,\tilde{v}_i\big), \tag{9}$$

with the 1.4826 consistency factor recovering a Gaussian σ from the MAD and `raim_sigma0_floor` (default 1.0) flooring it. Unlike the Kalman variance factor of Equation (6), $\hat{\sigma}_{0,\mathrm{net}}$ is emphatically *not* reporting-only: it drives a within-epoch RAIM adaptation loop (refloat satellite / exclude station) whenever it exceeds `network.raim_sigma0_trigger` (default 3.0), a whole-epoch and per-constellation product withhold above `network.raim_withhold_sigma0`, and the scale on the emitted URA (`ApplicationNetwork.cpp`; `NetworkEstimator::decidePerConstWithhold`). The full producer-side RAIM control loop is out of scope here; only the estimator Equation (9) and its redundancy input (Section 1.3) are.

1.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{10}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance, so that $\boldsymbol{Q}$ is the covariance of $\hat{\boldsymbol{v}}$ propagated through the gain. Its diagonal $\boldsymbol{Q}_{ii}$ is built per row in `src/core/filter/KalmanFilter.cpp` (line 920) as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$; the full matrix is formed only under the off-diagonal kernel of Section 1.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{11}$$

clamped to $[0,1]$ for numerical safety (line 1774 in the diagonal path, line 1757 in the full-matrix path). $r_i \to 1$ means the observation is fully checked by the geometry (a blunder lands almost entirely in the residual); $r_i \to 0$ means it is *uncheckable* (a blunder is absorbed into the estimate and leaves no residual signature). The sum dof $= \sum_i r_i = \operatorname{tr}\boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (6).

The network producer reimplements Equation (11) in the batch/arrowhead algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i\,\boldsymbol{a}_i^{\mathrm{T}}\operatorname{Cov}(\boldsymbol{x})\,\boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\,\boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i}\,]$ and the same $[0,1]$ clamp — algebraically the batch twin of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows falling below a 10^{-6} redundancy floor are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (9) as uncheckable — the network analogue of the KF snooping gate's 0.1 cutoff below, at a much smaller constant because it guards only the $\sqrt{r_i}$ standardisation, not a reset decision.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{12}$$

(line 1780), which — crucially — divides by $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not by $\sqrt{R_{ii}}$. This is the Studentised residual proper; under the null hypothesis $w_i \sim \mathcal{N}(0,1)$, whereas the s used by the fixed gate floors the denominator at R and is therefore conservative. When the residual covariance is non-diagonal the correlated Teunissen generalisation is used instead (line 1761):

$$\boxed{w_i \;=\; \frac{\big(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{\big(\boldsymbol{Q}^{-1}\big)_{ii}}}.} \tag{13}$$

Equation (13) reduces to Equation (12) when $\boldsymbol{Q}$ is diagonal, but accounts for the off-diagonal correlation among residuals introduced by differencing [2, 3].

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{14}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (lines 1764–1765, 1781–1782). The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$ (the 20% false-negative quantile, i.e. $\beta = 0.20$ power); for the default critical $k = 3.29$ this gives $\delta_0 \approx 4.13$ (line 1728). The $1/\sqrt{r_i}$ factor is the geometric leverage: a poorly-checked observation ($r_i \to 0$) has an arbitrarily large MDB — the code returns $+\infty$ when r_i falls below a small floor — so a blunder there is by construction undetectable from the residual alone. The MDB is a Kalman-filter QC quantity only: the network producer implements no minimal detectable bias, relying instead on the robust $\hat{\sigma}_{0,\text{net}}$ RAIM loop of Equation (9).

1.4 Data-snooping gate override

The QC statistics are not merely reported; when `enableDataSnooping()` is set they drive an *alternative, parallel* outlier gate that can reselect the row the fixed $|s| > 6$ path missed. Both gates share the same downstream dispatch in `src/core/filter/KalmanFilter.cpp`; only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$ so the dispatch is byte-identical when snooping is off (lines 2919–2921).

When snooping is enabled the gate is instead reseeded from the w-test over *checkable* rows only (lines 2922–2940): rows with $r_i < 0.1$ are skipped as uncheckable (lines 1958, 2929), because their residual is unreliable and resetting the corresponding ambiguity would discard good information. Over the surviving rows the gate fires on

$$\max_{\{i\,:\,r_i \geq 0.1\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{15}$$

and the worst checkable $|w_i|$ row becomes the handled index (lines 2936–2940). Because w_i uses the true residual σ (Equation (12)) rather than the R-floored s, this gate can catch a blunder on a high-leverage row whose floored s stayed below 6, and it overrides the s pick (lines 2963–2968): the downstream Melbourne–Wübbena tiebreaker still operates in the σ domain, so when the w-test selected a row the σ cut missed, no σ-candidate is collected and the w-selected index is handled directly. This is the complement of the fixed standardised-residual path in the gating section, which this document does not re-derive.

1.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which holds for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of the differencing transformations maps the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\text{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\text{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$, carrying the structured SD/DD cross-covariance between observations that share a pivot or reference. Under the off-diagonal kernel (`src/core/filter/KalmanFilter.cpp` lines 1743–1766) the full residual covariance Equation (10) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \text{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{16}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{Q}^{-1})_{ii}}}, \qquad \text{MDB}_i = \frac{\sqrt{R_{ii}}\,\delta_0}{\sqrt{r_i}}. \tag{17}$$

$\boldsymbol{R}^{-1}$ and $\boldsymbol{Q}^{-1}$ are obtained by LDL^{T} solves against the identity (lines 1748–1752), exploiting the symmetric positive-definiteness of both. The kernel is consistent by construction: the diagonal of the full $\boldsymbol{\Theta}$ and of the full Teunissen w equal the per-row quantities of Section 1.3 whenever the off-diagonals vanish, so the *outlier-handling path is unchanged behind the flag* — it only gains

correctness on differenced epochs where the residual correlations are real. This is the measurement-side counterpart to the structured correlation the differencing section introduces on the model side.

References

[1] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[3] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.